API CONTRACT DRAFT

ELKash ERP System

Version: V1 Draft
Architecture: REST API
Authentication: Laravel Sanctum
API Base URL:

```
/api/v1
```

---

1. API PRINCIPLES

API Design Standards

ELKash API harus:

- RESTful
- modular
- versioned
- predictable
- typed-response ready
- scalable for future ERP modules

Response Standard

Success Response

```
{
  "success": true,
  "message": "Transaction created successfully",
  "data": {}
}
```

Error Response

```
{
  "success": false,
  "message": "Stock unavailable",
  "errors": {
    "product_id": [
      "Selected product stock is insufficient"
    ]
  }
}
```

---

2. AUTHENTICATION API

Authentication Flow

```
Nuxt Frontend
↓
Laravel Sanctum
↓
Session/Cookie Authentication
↓
Protected API Access
```

2.1 Login

Endpoint

```
POST /api/v1/auth/login
```

Request Body

```
{
  "email": "cashier@elkash.com",
  "password": "password123"
}
```

Validation Rules

Field	Rule
email	required, email
password	required, min:8

Success Response

```
{
  "success": true,
  "message": "Login success",
  "data": {
    "user": {
      "id": 1,
      "name": "Cashier User",
      "email": "cashier@elkash.com",
      "role": {
        "id": 3,
        "name": "Cashier"
      }
    },
    "permissions": [
      "pos.transaction.create",
      "pos.transaction.view"
    ]
  }
}
```

Error Response

```
{
  "success": false,
  "message": "Invalid credentials",
  "errors": {}
}
```

2.2 Logout

Endpoint

```
POST /api/v1/auth/logout
```

Authentication

Required

Success Response

```
{
  "success": true,
  "message": "Logout success",
  "data": null
}
```

2.3 Current Authenticated User

Endpoint

```
GET /api/v1/auth/me
```

Success Response

```
{
  "success": true,
  "message": "Authenticated user",
  "data": {
    "id": 1,
    "name": "Admin",
    "email": "admin@elkash.com",
    "role": "Admin"
  }
}
```

---

3. PRODUCT API

3.1 Get Products

Endpoint

```
GET /api/v1/products
```

Query Parameters

Parameter	Type	Description
search	string	search keyword
category_id	integer	filter category
is_available	boolean	filter visibility
page	integer	pagination page
limit	integer	item per page

Success Response


```
{
  "success": true,
  "message": "Products fetched",
  "data": {
    "items": [
      {
        "id": 1,
        "sku": "PRD-001",
        "name": "Cappuccino",
        "price": 25000,
        "stock_quantity": 30,
        "is_available": true,
        "category": {
          "id": 2,
          "name": "Beverages"
        }
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 10,
```

```
      "total": 100
    }
  }
}
```

3.2 Create Product

Endpoint

```
POST /api/v1/products
```

Authorization

Admin / Super Admin

Request Body

```
{
  "category_id": 1,
  "supplier_id": 1,
  "sku": "PRD-001",
  "name": "Americano",
  "description": "Hot americano coffee",
  "price": 22000,
  "cost_price": 12000,
  "stock_quantity": 100,
  "minimum_stock": 10,
  "is_available": true
}
```

Validation Rules

Field	Rule
sku	required, unique
name	required
price	required, numeric
stock_quantity	required, integer, min:0

Success Response

```
{
  "success": true,
  "message": "Product created",
  "data": {
    "id": 1
  }
}
```

3.3 Update Product

Endpoint

```
PUT /api/v1/products/{id}
```

Business Rules

- historical transactions must remain preserved
- soft delete preferred
- immutable transaction snapshots

3.4 Delete Product

Endpoint

```
DELETE /api/v1/products/{id}
```

Business Rules

- soft delete only
- transaction history must remain intact

---

4. CATEGORY API

4.1 Get Categories

Endpoint

```
GET /api/v1/categories
```

Success Response

```
{
  "success": true,
  "message": "Categories fetched",
  "data": [
    {
      "id": 1,
      "name": "Coffee"
    }
  ]
}
```

---

5. INVENTORY API

5.1 Stock Adjustment

Endpoint

```
POST /api/v1/inventory/adjustments
```

Authorization

Admin / Super Admin

Request Body

```
{
  "product_id": 1,
```

```
  "movement_type": "ADJUSTMENT",
  "quantity": 20,
  "notes": "Initial stock correction"
}
```

Business Rules

- stock movement mandatory
- audit log mandatory
- negative stock prohibited

Success Response

```
{
  "success": true,
  "message": "Stock adjusted successfully",
  "data": {
    "product_id": 1,
    "previous_stock": 80,
    "current_stock": 100
  }
}
```

5.2 Get Stock Movements

Endpoint

```
GET /api/v1/inventory/stock-movements
```

Query Parameters

Parameter	Description
product_id	filter by product
movement_type	filter type
start_date	date range
end_date	date range

---

6. POS TRANSACTION API

6.1 Create Transaction

Endpoint

```
POST /api/v1/transactions
```

Authorization

Cashier / Admin

Request Body

```
{
  "items": [
    {
      "product_id": 1,
      "quantity": 2
    },
    {
      "product_id": 2,
      "quantity": 1
    }
  ],
  "payment": {
    "payment_method": "CASH",
    "paid_amount": 100000
  }
}
```

Transaction Integrity Rules

- inventory deduction inside DB transaction
- rollback mandatory on failure
- duplicate checkout prohibited
- failed payment must not reduce stock
- completed transaction immutable

Success Response

```
{
  "success": true,
  "message": "Transaction created",
  "data": {
    "transaction_id": 1001,
    "transaction_code": "TRX-20260528-0001",
    "grand_total": 75000,
    "change_amount": 25000,
    "invoice_url": "/invoice/TRX-20260528-0001"
  }
}
```

Stock Error Response

```
{
  "success": false,
  "message": "Stock unavailable",
  "errors": {
    "items.0.quantity": [
      "Insufficient stock"
    ]
  }
}
```

6.2 Get Transaction History

Endpoint

```
GET /api/v1/transactions
```

Query Parameters

Parameter	Description
cashier_id	filter cashier
payment_status	filter payment
start_date	filter date

Parameter	Description
end_date	filter date

6.3 Get Transaction Detail

Endpoint

```
GET /api/v1/transactions/{id}
```

6.4 Cancel Transaction

Endpoint

```
POST /api/v1/transactions/{id}/cancel
```

Business Rules

- completed transaction requires authorization
- audit log mandatory
- inventory rollback required

---

7. PAYMENT API

7.1 Payment Methods

Supported methods:

- CASH
- QRIS
- DEBIT
- CREDIT_CARD
- E_WALLET

7.2 Process Payment

Endpoint

```
POST /api/v1/payments
```

Request Body

```
{
  "transaction_id": 1001,
  "payment_method": "QRIS",
  "paid_amount": 75000,
  "payment_reference": "QRIS-INV-2026"
}
```

---

8. DASHBOARD API

8.1 Dashboard Summary

Endpoint

```
GET /api/v1/dashboard/summary
```

Success Response

```
{
  "success": true,
  "message": "Dashboard summary fetched",
  "data": {
    "today_revenue": 2500000,
    "today_transactions": 124,
    "low_stock_count": 5,
    "best_seller": [
      {
        "product_name": "Americano",
        "sold_quantity": 80
      }
    ]
```

```
  }
}
```

---

9. REPORTING API

9.1 Daily Sales Report

Endpoint

```
GET /api/v1/reports/daily
```

Query Parameters

Parameter	Description
date	report date

9.2 Monthly Sales Report

Endpoint

```
GET /api/v1/reports/monthly
```

9.3 Inventory Report

Endpoint

```
GET /api/v1/reports/inventory
```

9.4 Cashier Report

Endpoint

```
GET /api/v1/reports/cashier
```

9.5 Export Report

Endpoint

```
POST /api/v1/reports/export
```

Request Body

```
{
  "report_type": "daily_sales",
  "format": "PDF",
  "filters": {
    "date": "2026-05-28"
  }
}
```

Supported Formats

- PDF
- Excel

---

10. USER MANAGEMENT API

10.1 Get Users

Endpoint

```
GET /api/v1/users
```

10.2 Create User

Endpoint

```
POST /api/v1/users
```

Request Body

```
{
  "name": "Cashier A",
  "email": "cashier@elkash.com",
  "password": "password123",
  "role_id": 3
}
```

10.3 Update User Role

Endpoint

```
PUT /api/v1/users/{id}/role
```

---

11. RBAC API

11.1 Get Roles

Endpoint

```
GET /api/v1/roles
```

11.2 Get Permissions

Endpoint

```
GET /api/v1/permissions
```

---

12. AUDIT LOG API

12.1 Get Audit Logs

Endpoint

```
GET /api/v1/audit-logs
```

Authorization

Super Admin

Query Parameters

Parameter	Description
module	filter module
user_id	filter actor
start_date	filter date
end_date	filter date

---

13. HTTP STATUS STANDARD

Status Code	Meaning
200	Success
201	Resource created
400	Validation/business error

Status Code	Meaning
401	Unauthorized
403	Forbidden
404	Resource not found
409	Conflict
422	Validation failed
500	Internal server error

---

14. SECURITY RULES

Backend Security

- frontend never directly accesses database
- all business validation handled in Laravel
- RBAC validated server-side
- protected endpoints require Sanctum session
- sensitive actions require authorization policies

---

15. FUTURE API PREPARATION (V2)

Reserved future modules:

```
/api/v1/purchasing
/api/v1/purchase-requests
/api/v1/purchase-orders
/api/v1/sales-orders
/api/v1/approval-flows
```

---

16. ENGINEERING PRINCIPLES

ELKash API prioritizes:

1. data consistency
2. transaction reliability
3. predictable API response

4. maintainable backend architecture
5. scalable modular structure

Avoid:

- overengineered API abstraction
- premature microservices
- inconsistent response format
- business logic leakage to frontend